

DEFINING CLOUD COMPUTING

Meaning of Cloud Computing

Cloud computing is a model where computing resources (such as software, storage, servers, and services) are provided on demand, similar to utilities like electricity or water.

Instead of owning and managing physical hardware, users can access resources whenever needed and pay only for what they use.

Cloud computing turns technology, services, and applications into a self-service utility.

Two Essential Concepts of Cloud Computing

Cloud computing is based on two core concepts:

1. Abstraction

Abstraction means hiding the internal working details from users.

In cloud computing:

- Users do not know where the servers are located
- Users do not manage hardware or system maintenance
- Applications run on unspecified physical machines
- Data is stored in unknown physical locations
- System administration is handled by cloud providers
- Services can be accessed from anywhere via the internet

👉 User focuses only on usage, not implementation.

2. Virtualization

Virtualization means creating virtual (logical) resources from physical hardware.

In cloud computing:

- Physical resources are pooled together

- Virtual servers and storage are created on demand
- Multiple users share the same infrastructure (multi-tenancy)
- Resources can be scaled up or down quickly
- Usage is metered, so users pay based on consumption

👉 Resources are shared, flexible, and cost-efficient.

Computing as a Utility (Basic Idea)

The idea of computing as a utility existed from the early days of computing.

Earlier:

- Computing resources were expensive
- Systems were fixed and inflexible

Now:

- New technologies + need for efficiency
- Enabled on-demand, affordable, and scalable computing

Cloud computing makes it possible to:

- Use resources only when required
- Avoid heavy upfront investment
- Deploy applications faster

Cloud Computing vs Internet (Common Confusion)

Many people think cloud computing is just another name for the Internet, but this is not fully correct.

Why the confusion exists:

- Internet diagrams are often drawn as a cloud
- Applications running online are called “running in the cloud”

Internet already provides:

- Abstraction
- Standard protocols

- Common operating systems and applications

Even large intranets (internal company networks) appear like a cloud when systems become too complex to draw individually.

What Makes Cloud Computing Different

Cloud computing is more than just the Internet.

It is:

- An abstraction of pooled physical resources
- A new way to provision computing resources
- A model for:
 - Application deployment
 - Platform-independent access
 - On-demand scalability

Clouds can be: Public or private, Provider-managed or self-managed.

👉 Therefore, it is important to identify the type of cloud system being used.

Examples of Cloud Computing in Practice

1. Google (Software as a Service – SaaS)

Google built a global network of data centers. Provides services like search and free applications. Earns revenue mainly through advertising. Offers software directly to users via the internet.

👉 Example of Software as a Service (SaaS)

2. Microsoft Azure Platform (Platform as a Service – PaaS)

Microsoft Azure allows developers to run applications online. Supports .NET Framework applications. Acts as an alternative to desktop-based development platforms.

👉 Example of Platform as a Service (PaaS)

3. Amazon Web Services (Infrastructure as a Service – IaaS)

Amazon Web Services allows users to: Rent virtual machines, Rent storage and networking resources. Infrastructure is owned and managed by Amazon.

👉 Example of Infrastructure as a Service (IaaS)

Importance of Cloud Computing

Cloud computing enables: Low-cost application development, Fast deployment, Global availability, Easy scalability based on demand.

This represents a major shift in enterprise computing, changing how systems are: Built, Deployed, Managed, Scaled.

CLOUD TYPES

Need for Cloud Classification

To understand cloud computing clearly, it is important to classify cloud systems properly. Since cloud computing uses many new terms and acronyms, a standard classification helps avoid confusion.

Most experts divide cloud computing into two main models:

- Deployment Models
- Service Models

This separation is widely accepted and used in exams and industry.

Two Major Cloud Models

1. Deployment Models

Deployment models describe: Where the cloud is located, Who owns and manages the infrastructure.

👉 Focus is on infrastructure location and control.

2. Service Models

Service models describe: What type of services are provided, How much responsibility the user vs provider has.

👉 Focus is on type of service offered to users.

The NIST Cloud Computing Model

The National Institute of Standards and Technology (NIST) provides official definitions for cloud computing.

NIST divides cloud computing into: Deployment models, Service models, Essential characteristics.

This model is commonly used in: Textbooks, Research papers, University exams.

NIST and Virtualization / Multi-Tenancy

Earlier NIST definitions: Did not strictly require virtualization, Did not strictly require multi-tenancy.

Latest NIST definition: Requires virtualization, Requires multi-tenancy.

Multi-tenancy

Multi-tenancy means: Multiple customers share the same cloud resources, Each customer's data and access are kept logically separate.

Essential Characteristics of Cloud Computing (NIST)

According to NIST, cloud computing has the following key features:

- **On-Demand Self-Service:** Users can provision resources automatically without human interaction.
- **Broad Network Access:** Services are accessible through standard networks (Internet).
- **Resource Pooling:** Provider pools resources to serve multiple users.
- **Rapid Elasticity:** Resources can be quickly scaled up or down.
- **Measured Service:** Usage is monitored and billed on a pay-as-you-go basis.

Limitations of the NIST Model

The NIST model focuses mainly on core cloud services. Does not fully include: Service brokers, Integration services, Interoperability services, Cloud APIs. As cloud systems evolve, future models may include these components.

The Cloud Cube Model

The Cloud Cube Model was proposed by the Jericho Forum (The Open Group).

Purpose: To classify cloud environments, To understand security boundaries in cloud systems.

This model shows that traditional firewall boundaries no longer apply clearly in cloud computing.

Four Dimensions of the Cloud Cube Model

1. **Physical Location of Data:** Internal (I) → Data inside organization, External (E) → Data outside organization
2. **Ownership:** Proprietary (P) → Owned by one organization, Open (O) → Shared, interoperable systems
3. **Security Boundary:** Perimeterised (Per) → Inside firewall, De-perimeterised (D-p) → Outside firewall
4. **Sourcing:** Insourced → Managed by organization, Outsourced → Managed by cloud provider

👉 This model highlights that security and control vary greatly depending on cloud type.

Deployment Models of Cloud Computing (NIST)

A deployment model defines: Purpose of the cloud, Location of infrastructure, Ownership and management. NIST defines four deployment models.

1. Public Cloud

Definition: A public cloud is available for public use and owned by a cloud service provider.

Features: Owned and managed by provider, Used by general public or large groups, Cost-effective and scalable.

Example: Amazon Web Services

2. Private Cloud

Definition: A private cloud is used exclusively by one organization.

Features: Greater security and control, Can be managed internally or by third party, Can be On-premises or Off-premises.

3. Hybrid Cloud

Definition: A hybrid cloud combines two or more clouds (public, private, or community).

Features: Each cloud retains its identity, Data and applications can move between clouds, Supports flexibility and portability.

4. Community Cloud

Definition: A community cloud serves a specific group of organizations with shared requirements.

Features: Shared mission or policies, Common security or compliance needs, Managed by community or third party.

Example: U.S. government cloud platforms

Service Models of Cloud Computing

Service models describe what level of service is provided. All service models follow the pattern: XaaS – “Something as a Service”. The three universally accepted service models are called the SPI Model.

[Image of IaaS PaaS SaaS Pyramid]

SPI Model of Cloud Computing

1. Infrastructure as a Service (IaaS)

Definition: IaaS provides virtualized hardware resources.

Includes: Virtual machines, Storage, Networks.

Responsibility: Provider → Infrastructure, User → OS, applications, data.

Example: Amazon EC2

2. Platform as a Service (PaaS)

Definition: PaaS provides a development platform for building and deploying applications.

Includes: OS, Development tools, Frameworks.

Responsibility: Provider → Infrastructure + OS + platform, User → Applications.

Examples: Google App Engine, Microsoft Azure

3. Software as a Service (SaaS)

Definition: SaaS provides ready-to-use applications over the internet.

Features: Accessed via browser, No installation or maintenance by user.

Responsibility: Provider → Everything, User → Data and usage.

Examples: Salesforce, Google Apps

Relationship Between Service Models

- IaaS → Lowest abstraction
- PaaS → Built on IaaS
- SaaS → Built on PaaS + IaaS

👉 Each higher level inherits services from the lower level.

EXAMINING THE CHARACTERISTICS OF CLOUD COMPUTING

Introduction

Cloud computing is built on many existing computing concepts, such as networking, virtualization, distributed systems, and the Internet. Because of this, beginners may feel that cloud computing is not something new.

However, cloud computing represents a paradigm shift in how computing services are: Produced, Delivered, Consumed. It is an evolutionary change in technology, but it enables a revolutionary change in usage and business models.

Paradigm Shift in Computing

A paradigm shift means a fundamental change in thinking or approach.

In cloud computing: Users no longer buy hardware, Users rent or lease computing resources, Infrastructure is shared and centrally managed.

When you choose a cloud provider, you are effectively renting a portion of a massive global infrastructure consisting of: Datacenters, Servers, Storage systems, Networking equipment.

Scale of Cloud Infrastructure

Modern cloud infrastructure is built on a huge industrial scale.

Key observations:

- Cloud datacenters cost around \$100 million each
- Semiconductor fabrication plants cost \$2–5 billion
- Cloud providers operate multiple datacenters worldwide

Examples:

Amazon operates dozens of datacenters globally. Google operates 30+ large-scale datacenters. 👉 This scale allows cloud providers to offer computing as a utility service, similar to electricity.

Historical Background

1960s – Hardware Foundations: Military-funded research helped miniaturize electronics. Led to: Microprocessors, Dense memory, Integrated circuits.

1990s – Internet Commercialization: Growth of Internet-based businesses. Companies like Amazon and Google required huge computing capacity. Large-scale datacenters became necessary.

Amazon and Elastic Infrastructure

Amazon built its infrastructure to handle peak demand, especially during events like: Black Friday, Holiday sales.

Key idea: Infrastructure was idle during non-peak periods. Amazon opened this unused capacity to: Partners, External customers. This led to the creation of Amazon Web Services (AWS).

Google and Global Datacenters

Google's services (Search, Email, Maps, Video streaming) Required: Massive processing power, Global datacenters.

Some datacenters are as large as: An American football field. These datacenters act like industrial-scale IT utilities.

Greenfield Datacenters

As datacenters grew larger, companies began building greenfield datacenters (built from scratch).

These datacenters are located to: Access low-cost electricity, Use renewable energy, Be near abundant water, Connect to high-speed network backbones, Reduce land and operational costs, Optimize network latency, Gain tax incentives.

👉 These factors make cloud computing highly efficient and profitable.

Cloud Computing as an IT Utility

It is estimated that: The Internet consumes ~10% of global electricity. Cloud companies: Are major energy consumers, May also become major energy producers.

Result: The Internet has effectively funded the world's first IT utilities. Cloud computing treats computing like a public utility.

Popular Cloud Applications (IDC Study)

According to IDC, top cloud application areas include: Collaboration applications, Web applications and hosting, Cloud backup services, Business applications, Personal productivity applications.

Everyday Impact of Cloud Computing

Many applications previously deployed as On-premises or Client/server systems Have now moved to the cloud without users noticing.

Examples: CRM systems, Call center services, Online productivity tools.

This transition has been: Transparent, Gradual, Seamless.

Shift in Software Delivery Economics

Cloud computing has changed software delivery in the same way that Digital downloads changed music distribution.

Key changes: No physical packaging, No retail shelf space, Lower distribution costs, Subscription or pay-as-you-go pricing.

This allowed: New vendors to enter the market, Lower prices for users, Free services supported by advertising.

Essential Characteristics of Cloud Computing (NIST)

1. **On-Demand Self-Service:** Users can provision resources automatically, No human interaction with provider required, Faster deployment and flexibility.
2. **Broad Network Access:** Services accessed via standard networks. Works across: Laptops, Mobile phones, Tablets. Platform and OS independent.
3. **Resource Pooling:** Provider pools physical and virtual resources. Supports multi-tenancy. Resource location is hidden from users. Resources are allocated dynamically.

4. **Rapid Elasticity:** Resources can be scaled Up (scale-up) or Out (scale-out). Scaling can be automatic or manual. From the user's view, resources appear unlimited.
5. **Measured Service:** Resource usage is monitored and measured. Users pay only for what they consume. Billing based on: Storage, Bandwidth, Processing power, Transactions.

Additional Advantages of Cloud Computing

- **Lower Costs:** Higher efficiency and shared resources reduce expenses.
- **Ease of Use:** Minimal hardware and software requirements.
- **Quality of Service (QoS):** Performance and availability guaranteed by contracts.
- **High Reliability:** Load balancing and failover improve uptime.
- **Outsourced IT Management:** Reduces need for in-house IT staff.
- **Simplified Maintenance:** Centralized updates and patches.
- **Low Barrier to Entry:** Very low initial investment; easy startup.

Disadvantages of Cloud Computing

Despite many benefits, cloud computing has limitations.

1. **Limited Customization:** Cloud applications are standardized, Less flexible than on-premises software, Advanced features may be missing.
2. **Latency Issues:** Data travels over WAN, High latency affects: Data-intensive applications, Real-time systems.
3. **Stateless Architecture Overhead:** Cloud systems are stateless by design. Requests are independent (HTTP GET, PUT, etc.). Maintaining transaction consistency requires: Middleware, Brokers, Transaction managers. 🖐️ This adds performance overhead.
4. **Security and Privacy Risks:** Data is stored outside user control. Risk of: Data interception, Unauthorized access, Government surveillance. Sensitive data requires extra caution.
5. **Regulatory and Compliance Issues:** Organizations must follow laws such as Accounting regulations, Health data privacy laws, Data residency laws. Challenges: Different laws in different countries, Data at rest vs data in transit. Responsibility lies mainly with the client. Cloud compliance is still complex and evolving.

ASSESSING THE ROLE OF OPEN STANDARDS IN CLOUD COMPUTING

Introduction

Cloud computing has not emerged from a single technology. Instead, it is the result of the convergence of many existing standards in computing, networking, and software architecture.

The key promise of cloud computing is scalability, which changes how: Applications are developed, Services are deployed, Infrastructure is consumed.

Without open standards, cloud computing risks becoming a collection of proprietary systems, leading to vendor lock-in.

Need for Open Standards

Vendor Lock-in Problem

When cloud providers use proprietary technologies: Customers become dependent on a single vendor, Migration to another provider becomes difficult, Data portability is limited.

Such isolated systems are often called “stovepipe clouds”.

👉 Since customers want flexibility and freedom of choice, the industry strongly supports standards-based cloud platforms.

Architectural Standards Supporting Cloud Computing

Cloud computing relies on several well-established architectural standards:

1. **Platform Virtualization of Resources:** Abstracts hardware into virtual machines, Enables efficient resource sharing, Forms the foundation of IaaS.
2. **Service-Oriented Architecture (SOA):** Applications are built as reusable services, Services communicate using standard interfaces, Improves interoperability and flexibility.
3. **Web Application Frameworks:** Support scalable, browser-based applications, Enable rapid development and deployment, Used heavily in SaaS and Web 2.0 applications.
4. **Open-Source Software Deployment:** Encourages transparency and collaboration, Reduces licensing costs, Avoids dependency on proprietary vendors.
5. **Standardized Web Services:** Use common protocols (HTTP, REST, SOAP), Enable communication between heterogeneous systems, Improve cross-platform compatibility.
6. **Autonomic Systems:** Systems that can self-manage, Include self-configuration, self-healing, self-optimization, Reduce human intervention.

7. **Grid Computing:** Combines distributed computing resources, Supports large-scale processing tasks, Influenced modern cloud resource pooling.

Importance of Open Standards for Cloud Business Models

These standards support major cloud business models such as: Software as a Service (SaaS), Web 2.0 applications, Utility computing.

For these models to succeed: Data must be portable, Services must be universally accessible, Applications must be interoperable across platforms.

👉 Open standards are essential for long-term cloud adoption.

Proprietary vs Open Cloud Platforms

Currently, many cloud platforms are proprietary, offered by vendors such as: Microsoft (Azure Platform), VMware (vSphere). To counter this, the industry is actively developing open cloud platforms.

OpenStack Project

In July 2010, Rackspace announced an open-source cloud project called OpenStack.

Goals of OpenStack: Create open cloud infrastructure software, Compete with proprietary cloud platforms, Prevent vendor lock-in.

Contributions and Licensing: Rackspace contributed code from: Cloud Files, Cloud Servers. NASA contributed technology from its Nebula Cloud Platform. Released under the Apache 2.0 open-source license.

Founding Members: Major contributors include: AMD, Citrix, Dell, Intel, NTT Data, Other cloud service providers. This broad support shows strong industry backing.

Core Components of OpenStack

1. **OpenStack Compute:** Automatically provisions large numbers of virtual servers, Runs on industry-standard hardware, Supports scalable IaaS environments.
2. **OpenStack Storage:** Provides redundant, object-based storage, Uses clusters of commodity servers, Ensures fault tolerance and scalability.

Eucalyptus Cloud Platform

Eucalyptus is: A Linux-based open-source IaaS platform, Designed for building private and hybrid clouds, Based on computer clusters.

Key Features: Compatible with Amazon EC2, Amazon S3. Allows developers to Use AWS APIs, Test applications in private cloud environments.

Virtualization Support: Eucalyptus works with: VMware, Xen, KVM. This makes it flexible and widely usable.

Origin and Commercialization: Originally developed by Rich Wolski at the University of California, Santa Barbara. Eucalyptus Systems was formed in 2009 to commercialize the platform. Supported by major Linux vendors.

Other Open Cloud Initiatives and Research

OpenStack and Eucalyptus are not the only efforts in this area. Additional developments include: Multiple open-source cloud platform projects, Academic and industrial research initiatives.

Role of IEEE in Cloud Standards

The IEEE: Supports cloud research through its Technical Committee on Services Computing, Organizes the CLOUD conference, Maintains working groups and publications on cloud computing.

These efforts help: Advance standardization, Encourage global collaboration, Improve cloud interoperability.

DEFINING INFRASTRUCTURE AS A SERVICE (IAAS)

Cloud Computing Ecosystem (Layered View)

Cloud computing can be broadly divided into four layers, which together form the cloud computing ecosystem:

1. Business Process Layer (SOA)
2. Application Layer
3. Platform Layer
4. Infrastructure Layer

Each layer supports the layer above it: Application layer → Basis for SaaS, Platform layer → Basis for PaaS, Infrastructure layer → Basis for IaaS.

Meaning of Infrastructure as a Service (IaaS)

Infrastructure as a Service (IaaS) is a cloud service model where: Hardware resources are virtualized, Computing infrastructure is delivered as a utility service, Users consume resources on demand, Users pay only for what they use.

IaaS allows users to tap into massive computing infrastructure without worrying about scalability limits.

IaaS as Utility Computing

IaaS works like a utility model: Similar to electricity or water, Resources are available when needed, No upfront capital investment, Billing is usage-based.

This makes IaaS a highly disruptive technology, because: Small businesses can scale rapidly, Large infrastructure is no longer a barrier, Growth can happen almost overnight. 👉 This capability has fueled many IaaS startups, even during economic recessions.

Ownership and Responsibility in IaaS

In the IaaS model:

Service Provider Owns: Physical servers, Storage systems, Network infrastructure, Datacenters.

User (Developer / Client) Controls: Virtual machines, Operating systems, Applications, Data, Services deployed on VMs.

👉 The provider offers hardware as a service, while the user builds software on top of it.

Virtualized Hardware Model

In IaaS: Physical hardware is abstracted, Users create virtual hardware, Virtual resources are mapped to real physical systems.

When a user requests resources: Request is made to a virtual system, Cloud redirects it to the actual physical servers, Physical servers perform the real computation.

Interaction with IaaS Resources

Developers use IaaS to create: Virtual private servers, Virtual private storage, Virtual private networks. They then: Install operating systems, Deploy applications, Configure services required for their solution.

IaaS Workloads

Meaning of a Workload: A workload is the fundamental unit of computation in IaaS. A workload: Simulates a physical server, Represents a certain processing capacity, Is defined by measurable performance metrics.

Workload Performance Metrics: A workload is measured using: Transactions Per Minute (TPM), Disk I/O (IOPS), RAM usage (MB or GB), Network throughput, Network latency.

Instances in Cloud Computing

In traditional hosting: Application runs on a dedicated physical server.

In IaaS: User provisions a server instance, An instance is a virtual server, Resources required for that server are dynamically allocated.

👉 An instance behaves like a real server but runs on shared infrastructure.

Instance Sizing and Partitioning

Different workloads require different server sizes: Small, Medium, Large.

To support this: IaaS providers partition resources, Allocate appropriate CPU, RAM, storage, and network capacity, Run instances from a shared pool of: Virtual machines, RAID storage, Network interfaces.

Logical vs Physical Layers in IaaS

IaaS architecture includes multiple logical constructs:

[Image of Infrastructure as a Service Architecture]

Logical Components: LUNs (Logical Unit Numbers) → Logical storage containers, Cloud interconnect layer → Virtual networking with assigned IP addresses, Virtual application software layer → Software running inside virtual machines.

Physical Components: Servers, Storage disks, Network hardware.

👉 Logical units are mapped onto physical assets transparently.

Networking Architecture in IaaS

Private Network Assignment

From an architectural standpoint: Each IaaS client is assigned a private virtual network. In Amazon Web Services EC2: Each instance behaves like it is on its own network, Scaling adds more logical networks, Traffic must be routed between these networks.

Virtual Private Cloud (VPC)

To overcome networking limitations: AWS provides Virtual Private Cloud (VPC). Allows better network control, Reduces some routing overhead.

However: Layer-2 networking (broadcast, multicast) is not supported, Network traffic must be routed at Layer-3.

Comparison with Other IaaS Providers

Some IaaS providers: Assign networks per user, Support Layer-2 networking. Examples: Traditional VMware-based clouds, Cloudscaling platforms.

Layer-2 networking allows: VLANs, Tunneling protocols, Greater flexibility in enterprise deployments.

Example: eCommerce Application Workloads

A typical transactional eCommerce system includes: Web server, Application server, File server, Database, Transaction engine.

Each component has different workloads: Database → query-intensive, Application server → logic processing, Web server → client request handling.

IaaS allows: Each workload to be sized independently, Resources to be scaled based on demand.

Amazon Web Services as a Classic IaaS Example

The most common example of IaaS is Amazon Web Services (AWS).

Key characteristics: Uses virtualization (Xen), Servers are partitioned into logical compute units, Multiple instance sizes are available.

Users can: Apply pre-built system images, Create custom images, Choose operating systems and applications. AWS also provides: Storage services, Databases, Messaging and notification services.

👉 This enables large-scale distributed applications.

Pods, Aggregation, and Scaling

Pods: A pod is a group of users supported by a single instance. Each instance can support only a limited load. When capacity is exceeded: Instance must be cloned, Additional instances are created.

Cloud Control System (CCS): Pods are managed by a Cloud Control System. In AWS: CCS is the AWS Management Console.

Availability Zones and Aggregation: Pods are grouped into availability zones. Availability zones: Exist within an IaaS region, Are typically aligned with datacenters, Provide fault isolation.

In AWS: Zones exist in regions such as: Northern California, Northern Virginia, Ireland, Singapore. If one zone fails: Other zones continue operating, Failover ensures high availability.

Silos in Cloud Computing

Meaning of a Silo: A silo is a cloud environment that: Is isolated from other clouds, Cannot easily interoperate, Acts as a closed processing domain. Silos are common in: PaaS platforms (e.g., Force.com, QuickBase), Certain IaaS network designs.

Pros and Cons of Silos:

- Advantages: Strong security, Better isolation, Easier to protect sensitive data.
- Disadvantages: Reduced interoperability, Limited flexibility, Risk of vendor lock-in.

👉 Silos are powerful but restrictive.

DEFINING PLATFORM AS A SERVICE (PAAS)

Meaning of Platform as a Service (PaaS)

Platform as a Service (PaaS) is a cloud service model that provides a complete software development environment in the cloud.

In PaaS: Developers can build, test, and deploy applications, Development happens within the tools and frameworks provided by the vendor, The platform hides all infrastructure and system-level complexity.

👉 PaaS focuses on application development, not hardware management.

Core Idea of PaaS

A PaaS platform provides: Programming languages, Application frameworks, Development tools, Runtime environments. Platforms may be based on: Specific programming languages, Specific frameworks, Vendor-defined APIs and services.

Most PaaS systems are delivered as a fully integrated development environment (IDE) in the cloud.

User Interface Support in PaaS

To be useful as a cloud service, PaaS must support user interface creation. Therefore, PaaS platforms support standards such as: HTML, JavaScript, Rich media technologies, Web UI frameworks.

This allows applications to be: Browser-based, Platform-independent, Accessible over the Internet.

Responsibility Model in PaaS

Customer Responsibilities: Interact with the application, Enter and retrieve data, Perform actions, Customize the application (within vendor limits).

Vendor Responsibilities: Hardware management, Operating systems, Platform software, Maintenance and upgrades, Application lifecycle management.

👉 The customer does not manage hardware or system software.

Google App Engine as a PaaS Example

The most commonly cited example of PaaS is Google App Engine. Key points: Developers program using Google APIs, File system structure and data storage are defined by Google, Development tools are tightly integrated, Applications run only within Google's environment.

Force.com as a PaaS Example

Another important PaaS platform is Force.com, provided by Salesforce. Key features: Acts as an add-on development environment, Allows developers to extend Salesforce's SaaS applications, Bridges SaaS and PaaS models.

Developer Perspective in PaaS

A developer may: Write applications using languages such as Python, Use vendor-specific APIs, Deploy applications entirely on the vendor's cloud.

In most cases: The PaaS vendor is also the solution provider, The customer consumes the finished application.

PaaS vs SaaS Overlap

Some services appear to be PaaS but are delivered as SaaS. Examples: Google Maps, Gmail, Google Earth. These: Follow PaaS principles internally, Are delivered to users as SaaS applications.

👉 Internally PaaS, externally SaaS.

Major Limitation of PaaS: Vendor Lock-in

The biggest drawback of PaaS is vendor dependency. Reasons: Applications use vendor-specific APIs, Code written for one platform may not run elsewhere, Migration to another platform is difficult.

Example: An application written for Google App Engine using Google APIs will only work in that environment.

👉 This creates significant vendor lock-in.

DEFINING SOFTWARE AS A SERVICE (SAAS)

Meaning of Software as a Service (SaaS)

Software as a Service (SaaS) is the most complete cloud service model. In SaaS: Hardware is provided by the vendor, Software is provided by the vendor, The complete solution is delivered as a service.

👉 SaaS is the cloud equivalent of shrink-wrapped software.

SaaS Definition (Simple)

SaaS is: Software hosted in the cloud, Accessed globally via the Internet, Usually through a web browser. All technical details are hidden from the user, except: User interaction with the application.

Familiar Examples of SaaS

Common SaaS applications include: Gmail, Google Calendar, QuickBooks Online, Zoho Office Suite.

SaaS also includes: Billing systems, CRM applications, Help Desk software, HR management systems, Online productivity tools.

Customization in SaaS

Many people believe SaaS is not customizable. This is: Mostly true for end-user tools (e.g., office suites), Not true for enterprise SaaS platforms.

Many SaaS platforms provide APIs that allow: Custom workflows, Custom data schemas, Integration with other systems, Security model changes.

Examples of customizable SaaS platforms: Salesforce, Quicken.

👉 SaaS does not always mean static software.

Key Characteristics of SaaS

All SaaS applications share these characteristics: Available globally via the Internet, Accessed on demand through a browser, Subscription-based or usage-based pricing, Vendor-managed software and infrastructure, Minimal client-side responsibility, Lower distribution and maintenance costs, Automatic upgrades and patching, Low barrier to entry and elastic scalability, Same software version for all users, Multi-tenant architecture with shared data model.

Shrink-Wrapped Software vs SaaS

Shrink-Wrapped Software: Locally installed, Owned by customer, Managed by local IT staff, One-time license + maintenance.

SaaS: Cloud-based, Subscription or metered pricing, Managed by cloud vendor, Governed by Service Level Agreements (SLAs).

Open SaaS and Open Source

When SaaS uses open-source software, it is called Open SaaS. Advantages: Lower deployment cost, Less vendor lock-in, Better portability, Large developer community. Common open-source components: Linux, Apache, MySQL, Perl (LAMP stack). Open SaaS improves: Profitability, Flexibility, Robustness of solutions.

SaaS and Service-Oriented Architecture (SOA)

A mature SaaS system often uses SOA: Applications are built from reusable services, Services communicate through standard protocols, Enterprise Service Bus (ESB) manages routing and integration. This architecture supports: Scalability, Interoperability, Component reuse.

Mashups in SaaS

Meaning of Mashups: A mashup is an application that: Combines data or features from multiple sources, Displays them in a single user interface. Example: Annotating Google Maps with external data. Mashups are a key feature of Web 2.0.

Components Required for Mashups: Interactive user interface (HTML, Ajax, JavaScript, CSS), Web services accessed via APIs (SOAP, REST, JSON), Data formats (XML, JSON, KML).

Open Mashup Alliance (OMA): Supports enterprise mashup technologies, Develops Enterprise Mashup Markup Language (EMML), Predicts rapid growth in mashup usage.

SaaS Market Growth

According to Gartner: ~25% of all software was expected to use the SaaS model by 2011. SaaS may be offered directly or via aggregators.

Aggregators: Combine SaaS applications from multiple vendors, Offer them as a unified solution.

Limits of SaaS Adoption

SaaS will not fully replace local software because: Risk of vendor shutdown, Internet dependency, Possible performance limitations.

However: SaaS pressures traditional software vendors to reduce prices, Drives competition and innovation.

Salesforce.com as SaaS and PaaS

Salesforce is a leading SaaS provider. Features: CRM, analytics, collaboration (Chatter), Sales, service, support, marketing tools. Founded in 1999 by former Oracle executives.

Force.com: SaaS to PaaS Transition: Salesforce extended SaaS into PaaS via Force.com: Developers build add-on applications, Uses Apex (Java-based language), UI built with XML, HTML, Ajax, Flex. Result: Thousands of applications, Hundreds of vendors, Strong SaaS–PaaS integration.

DEFINING COMPLIANCE AS A SERVICE (CAAS)

Introduction

Cloud computing operates across multiple geographical locations and legal jurisdictions. This creates a major challenge: The laws of the user's country, The laws of the data processing location, The laws of the service provider's country.

👉 These laws may not match, which makes legal compliance complex and risky.

Meaning of Compliance in Cloud Computing

Compliance means ensuring that: Data handling, Security practices, Privacy protection, Auditing and reporting all follow legal, regulatory, and industry standards.

Compliance is not just authentication or access control.

Why Compliance Is Complex

Compliance involves: Legal knowledge, Security expertise, Privacy management, Risk assessment, Continuous monitoring. Simply issuing an identity token or allowing access through SSO (Single Sign-On) is not sufficient.

👉 Compliance requires deep policy enforcement, not just identity verification.

What Is Compliance as a Service (CaaS)?

Compliance as a Service (CaaS) is a cloud service model where: Compliance responsibilities are handled by a trusted third party, Compliance checks are provided as a service, Customers rely on the provider to meet regulatory requirements.

CaaS acts as a man-in-the-middle service between: Users, Applications, Cloud resources.

Role of CaaS in Cloud Architecture

A CaaS system would need to: Be trusted by all parties, Often function as a separate layer in a Service Oriented Architecture (SOA), Intercept, evaluate, and validate requests before allowing access.

Because of this, CaaS must be: Highly secure, Auditable, Standards-compliant.

Responsibilities of a CaaS Provider

A full-fledged CaaS solution must be able to:

- **Manage Cloud Relationships:** Understand interactions between users, providers, and services.
- **Understand Security Policies:** Enforce organizational and regulatory security rules.
- **Administer Privacy Controls:** Ensure personal and sensitive data is handled legally.
- **Geographical Awareness:** Know where data is stored, processed, and transmitted.
- **Incident Response:** Detect, report, and respond to security or compliance violations.
- **Archiving and Auditing:** Maintain logs and evidence for compliance verification.
- **Query and Reporting Support:** Allow compliance checks and audits when required.
- **Service Level Agreement (SLA) Compliance:** All guarantees must be measurable and contractually defined.

👉 This makes CaaS difficult to implement, but extremely valuable.

Current State of CaaS

Although CaaS is discussed widely: Very few general-purpose CaaS products exist, Most implementations are industry-specific, CaaS is still an emerging service model.

Vertical Clouds and CaaS

To make compliance manageable, many providers build vertical clouds.

Vertical Clouds: Clouds designed for specific industries, Compliance requirements are well-defined, Easier to enforce regulations.

Examples of Vertical Clouds Offering CaaS

- **athenahealth:** Medical and healthcare industry, Focus on health data regulations
- **bankserv:** Banking and financial transactions
- **ClearPoint:** PCI-DSS compliance for payment card transactions
- **FedCloud:** Government-focused cloud compliance
- **Rackspace PCI-Compliant Cloud:** Compliance for merchant and enterprise workloads

CaaS and Private Clouds

CaaS is easier to implement in private clouds because: Data is controlled by a single organization, Security boundaries are clear, Auditing is simpler, Regulatory enforcement is stronger.

👉 As a result, most existing compliance systems use private clouds.

Value of Compliance as a Service

A well-implemented CaaS can: Measure compliance risks, Reduce legal exposure, Provide compliance guarantees, Indemnify customers against compliance failures.

This makes CaaS a high-value, premium cloud service.

Future Potential of CaaS

One possible future use of CaaS: Guarantee that emails meet regulatory standards, Act as a trusted digital postal service, Help reduce spam and fraudulent communication.

National or global networks could: Certify message authenticity, Enforce compliance automatically.

DEFINING IDENTITY AS A SERVICE (IDAAS)

Importance of Identity in Networks

Establishing and proving identity is a core function of any network. An identity service: Stores information about a digital entity, Allows that information to be queried and managed, Supports secure electronic transactions.

At its core, an identity service consists of: Data Store – stores identity information, Query Engine – retrieves identity data, Policy Engine – enforces rules and maintains data integrity.

Identity Challenges in Cloud Computing

In distributed systems such as Internetworks and Cloud computing environments, identity management becomes much harder because: The attack surface is larger, Requests come from many locations, Resources are shared across networks.

Secure systems depend on: Correct identity validation, Proper authorization based on identity.

👉 Identity is the foundation of trust in secure network design.

Meaning of Identity as a Service (IDaaS)

Identity as a Service (IDaaS) refers to: Providing identity management functions as a cloud service, Managing authentication, authorization, and identity data centrally.

Although the term IDaaS sounds new, identity services have existed since the early days of networking. Like many cloud terms, IDaaS is a modern label for long-existing services.

DNS as an Early IDaaS Example

A classic example of identity service is the Domain Name System (DNS). DNS: Maps domain names to IP addresses, Associates domains with owners, Stores metadata about identities.

Examples of DNS identity domains: .COM, .ORG, .EDU, .MIL, .TV, .RU

👉 DNS is an identity lookup service running at Internet scale, making it an early form of IDaaS.

Modern Definition of IDaaS (SOA-Based)

Today, experts define IDaaS more narrowly. Modern IDaaS must: Operate as a Service Oriented Architecture (SOA) component, Be standards-based, Interoperate with other services, Support federated systems.

This definition limits IDaaS to modern, cloud-native identity services rather than legacy systems.

What Is an Identity?

An identity is a set of characteristics that makes an entity recognizable. In computing, a digital identity includes: Attributes (data about the entity), Metadata, Relationships with other entities.

Key properties: An identity must be unique, It must exist within a defined context.

Identity and Database Concepts

The concept of identity closely matches: Entity–Relationship (ER) models, Object Role Modeling (ORM). Databases: Store identities as entities, Maintain relationships between entities. This makes identity management a fundamental database and system design problem.

Identity Profiles and Social Platforms

Identity can also be represented as a profile. Example: Social platforms like Facebook extend the idea of identity. They manage attributes, relationships, and behavior. 👉 This is another practical form of Identity as a Service.

Types of Identity Attributes (For Individuals)

An identity may include:

1. **Things You Are:** Biological traits, Age, gender, appearance, Biometrics (fingerprint, retina)
2. **Things You Know:** Passwords, PINs, Personal history, Educational background
3. **Things You Have:** Hardware tokens, Bank accounts, Security keys, Physical possessions
4. **Things You Relate To:** Family and friends, Licenses, Beliefs and habits, Online accounts and preferences

Authentication Methods

Single-Factor Authentication: Username + password. Least secure.

Two-Factor Authentication: Password + token (hardware or SMS). More secure.

Multi-Factor Authentication: Combines: Something you know, Something you have, Something you are. Requires external trust services.

👉 Cloud-based trust services are the most common IDaaS implementations today.

Non-Human Digital Identities

Not only users have identities. Other identities include: Machine accounts, Devices, Network interfaces, Services and applications.

Examples: MAC addresses uniquely identify network interfaces, Directory services store user and machine identities, Federated identity systems share identities across domains.

Windows Product Activation as Identity Service

Microsoft Windows Product Activation is a real-world identity system. It collects: Product key and product ID, GUID, Hardware attributes (CPU, BIOS, RAM, disk, NIC), MAC address, Regional and language settings.

These attributes are: Weighted, Combined into a unique identity profile. If enough attributes change: Re-activation is required. 👉 This is a proprietary cloud-based identity system.

Networked Identity Service Classes

Identity services used in networks include: Ticket/token services, Certificate authorities, Trust services, Federated identity providers. These services: Validate websites and users, Secure transactions, Enable access control.

Cost and Complexity of Identity Services

Identity protection: Is expensive, Requires high expertise, Generates a large amount of network traffic.

Facts: Hundreds of identity checks occur every minute, Every network packet carries identity-related headers. 👉
Identity traffic is a major portion of total network traffic.

Why IDaaS Is Valuable: Because Skilled security experts are scarce, Identity systems are complex, Organizations want cost efficiency. IDaaS becomes: Highly valuable, Cost-effective, Essential for distributed systems.

Services Provided by IDaaS

IDaaS may include: Authentication services, Directory services, Federated identity, Identity governance, Identity and profile management, Policies and role enforcement, Provisioning, Registration, Risk and audit monitoring, Single Sign-On (SSO).

Identity Relationships and Regulation

Sharing identity data: Is regulated by governments, Must be restricted to minimal disclosure, Depends on identity relationships. Only justified and trusted parties should access identity information.

Identity System Codes of Conduct

IDaaS systems must follow strict principles:

- **User Consent:** Users control their identity data.
- **Minimal Disclosure:** Share only what is necessary.
- **Justifiable Access:** Only trusted, authorized parties get access.
- **Directional Exposure:** Public identities: bidirectional, Private identities: unidirectional.
- **Interoperability:** Must work with other identity providers.
- **Unambiguous Human Identification:** Protect users from identity attacks.
- **Consistency of Service:** Simple, reliable, and consistent across platforms.

IDAAS INTEROPERABILITY

Meaning of IDaaS Interoperability

Identity as a Service (IDaaS) allows applications to: Use identity services without building them internally, Store identity data and logic externally, Integrate authentication and authorization with minimal development effort.

Because identity logic is externalized, applications can: Share identity services, Interoperate across systems, Support federated environments.

Role of SOA in IDaaS Interoperability

IDaaS applications are designed to comply with: Service Oriented Architecture (SOA) standards.

SOA compliance ensures that: Identity services are modular, Services can communicate using standard interfaces, Identity systems can interoperate and be federated.

👉 Without SOA compliance, IDaaS systems become isolated and non-scalable.

Need for Standards in IDaaS

For interoperability, cloud-based IDaaS systems must rely on industry standards. These standards ensure: Cross-vendor compatibility, Secure identity exchange, Policy enforcement, Auditing and accountability.

Core Standards Supporting IDaaS Interoperability

1. User-Centric Authentication (OpenID & Information Cards)

User-centric authentication allows: Users to control their identity, Applications to rely on trusted identity providers. Supported by: OpenID, Information Cards (CardSpace). These standards allow identity to be: Portable, Reusable, Federated across services.

2. XACML – Authorization Policy Language

XACML (eXtensible Access Control Markup Language) is a standard for: Writing authorization policies, Enforcing access control decisions. Key points: Policies are written in XML, Defines who can access what and under which conditions, Works closely with SAML.

Flow: SAML sends an authorization request → XACML evaluates the request against policies → Access is allowed or denied.

3. SPML – Service Provisioning Markup Language

SPML is an XML-based language used for: Integrating service provisioning, Managing identity-related resources. It is a standard of OASIS and: Conforms to SOA principles, Enables automated provisioning, Ensures identity data is entered once and reused.

4. XDAS – Distributed Audit Service

XDAS provides: Accountability for system access, Detection of security policy violations, Auditing of identity usage. It helps identify: Unauthorized users, Unauthorized actions by valid users.

IDaaS Standards Framework (Conceptual View)

An interoperable IDaaS system integrates:

- Authentication (OpenID, Information Cards)
- Authorization (XACML)
- Provisioning (SPML)
- Auditing (XDAS)

Together, these form a complete identity service framework.

Identity Governance Framework (IGF)

The Identity Governance Framework (IGF) is an initiative of the Liberty Alliance. Goals of IGF: Control identity information exchange, Enforce privacy policies, Support privacy auditing.

Standards used include: WS-Trust, ID-WSF, SAML, LDAP directory services.

CARML and Project Aristotle: In 2009, Liberty Alliance released: Client Attribute Requirements Markup Language (CARML), IGF Privacy Constraints. These formed the basis of Project Aristotle, an open-source effort to: Build APIs for identity interchange, Enforce privacy-aware identity sharing.

USER AUTHENTICATION STANDARDS

OpenID – User Authentication

OpenID is a standard for authenticating end users. How it works: User identity is stored by an OpenID provider, Identity is represented as a URL or XRI, Applications trust the provider's authentication.

Any application supporting OpenID can accept: A verified identity from a trusted provider.

OpenID Identity Providers

Major OpenID providers include: AOL, Facebook, Google, IBM, Microsoft, Yahoo!, PayPal, VeriSign, LiveJournal. Each provider: Authenticates the user, Controls the authentication method (passwords, tokens, biometrics). The standard is supported by the OpenID Foundation.

OpenID Single Sign-On (SSO)

Once authenticated: The user can access other OpenID-enabled sites, No need to log in again.

Process: User logs into trusted provider → Provider verifies identity → Identity is shared with relying party → Access is granted. User consent is usually required before sharing identity.

CardSpace – Information Card Authentication

CardSpace is a Microsoft identity client and part of the Identity Metasystem. Key features: Uses Information Cards (i-Cards), Built on OASIS WS-* standards, Uses encrypted, digitally signed tokens. Tokens are issued via a Security Token Service (STS).

CardSpace can authenticate users for: OpenID, SAML, Windows Live ID.

CardSpace Identity Selector

An Identity Selector: Stores digital identities locally, Displays them as visual Information Cards, Allows users to choose which identity to present. When selected: A signed XML token is issued, Token contains identity metadata, Token is validated by the service.

Support for CardSpace

CardSpace and Information Cards are supported by: IBM, Novell Through the Higgins Open Source Identity Framework, which: Uses i-Card metaphor, Provides vendor-neutral identity APIs, Manages digital identities via browser add-ons.

Identity File Formats

- **vCard:** Electronic business card format, Stores identity data, Used originally in email systems.
- **vCalendar / iCalendar:** Extends identity to scheduling, Standardized through IETF RFCs.

👉 These are interchange formats, not trust authorization systems.

Authorization Markup Languages

XML in Identity Systems

Cloud identity systems use: XML-based request and response messages, Self-describing schemas, Interoperable formats. Two key XML standards: XACML, SAML.

XACML Architecture

XACML defines access control using: PAP (Policy Administration Point) Manages policies, PDP (Policy Decision Point) Evaluates access requests, PEP (Policy Enforcement Point) Enforces decisions, PIP (Policy Information Point) Supplies additional data (e.g., LDAP, Active Directory).

A request results in an obligation that must be enforced.

SAML – Security Assertion Markup Language

SAML is used to: Exchange authentication and authorization information, Enable Single Sign-On (SSO).

A SAML assertion states: Who authenticated, When, Under what conditions.

Types of SAML queries: Authentication query, Attribute query, Authorization decision query. SAML messages: Are wrapped in SOAP, Transported over HTTP.

SAML Single Sign-On (SSO)

In SSO: Browser acts as user agent, Service provider requests authentication, Identity provider validates user, Authorization is returned, Access is granted. This allows secure cross-domain access.

SPML – Provisioning in IDaaS

Meaning of Provisioning: Provisioning includes: Preparing a resource, Granting access, Using the resource, Releasing it after use.

SPML Components: A provisioning system includes: Requesting Authority (RA) – client, Provisioning Service Point (PSP) – cloud service, Provisioning Service Target (PST) – resource being provisioned.

SPML ensures: Identity data entered once, Automated access management, Efficient cloud operations. Without SPML: Cloud systems become inefficient, Manual errors increase, Reliability decreases.